

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 16

KUBERNETES ADVANCED

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 16 — Kubernetes Advanced

Resource Management • HPA • Helm • RBAC • Production-Oriented Kubernetes

1. Session Overview

Session 15 introduced Helm as a way to package and parameterize Kubernetes resources. Session 16 moves from basic deployment to production-oriented Kubernetes: resource requests and limits, Horizontal Pod Autoscaling (HPA), Helm-based configuration, workload identity, and RBAC.

The central story is simple: a microservice must have enough resources, must not consume the whole cluster, must scale when demand increases, must be deployed repeatably, and must run with only the permissions it actually needs.

2. Learning Objectives

- Distinguish CPU/memory requests from limits.
- Explain the noisy-neighbor problem.
- Configure resource requests and limits for a Spring Boot service.
- Explain the HPA feedback loop: Metrics Server → HPA → Deployment replica count.
- Configure an HPA for product-service using CPU utilization.
- Understand minReplicas, maxReplicas, target utilization, and scale-down behavior.
- Understand why Metrics Server is required for the basic CPU-based HPA.
- Choose appropriately between raw manifests and Helm charts.
- Parameterize image, resources, replicas, and HPA settings in Helm.
- Explain ServiceAccounts, Roles, and RoleBindings.
- Apply least-privilege RBAC.
- Use namespaces for logical environment isolation.
- Connect all of these capabilities to GitOps and Argo CD.

3. Prerequisites

Students should already understand Pods, Deployments, Services, namespaces, kubectl, GitOps/Argo CD, and the basic Helm concepts from Sessions 14 and 15.

Examples use product-service from the Enterprise E-Commerce Platform and assume port 8080.

4. The Production Problem

Imagine product-service normally receives 20 requests per second. During a product launch, traffic becomes ten times higher.

- A Pod may consume excessive CPU and slow other workloads.
- A Pod may consume too much memory and be terminated with OOMKilled.
- The service may become slow before additional capacity is available.
- Kubernetes may not know how much CPU or memory the Pod needs for scheduling.
- Manually increasing replicas is slow and unreliable.

Kubernetes provides resource management and autoscaling mechanisms to address these problems.

5. Resource Requests vs Limits

Concept	Meaning	Purpose
Request	Amount Kubernetes uses when scheduling the Pod.	Scheduling/capacity planning
Limit	Maximum resource amount the container is allowed to consume.	Resource control

Simple rule: request answers 'How much should Kubernetes reserve for scheduling?' Limit answers 'How much may this container consume?'

6. CPU Requests and Limits

CPU is commonly expressed in millicores. 1000m is approximately one CPU core.

```
resources:
  requests:
    cpu: 250m
  limits:
    cpu: 500m
```

The request is used for scheduling. The limit constrains CPU consumption; exceeding a CPU limit can result in throttling.

7. Memory Requests and Limits

```
resources:
  requests:
    memory: 256Mi
  limits:
    memory: 512Mi
```

If a container exceeds its memory limit, it can be terminated and Kubernetes may report OOMKilled. Memory therefore needs careful sizing for JVM applications.

8. The Noisy Neighbor Problem

A noisy neighbor is a workload that consumes excessive shared CPU or memory and negatively affects other workloads.

```
product-service → excessive CPU/memory
order-service   → becomes slow
shared cluster  → unpredictable behavior
```

Requests and limits do not create new capacity, but they give Kubernetes better scheduling information and enforce resource constraints.

9. Spring Boot Resource Configuration

There is no universal correct resource value for every Spring Boot service. Start with measurements and load tests, then tune.

```
resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    cpu: 500m
    memory: 512Mi
```

These are teaching values, not production sizing recommendations.

10. Add Resources to a Deployment

```
containers:
- name: product-service
  image: ghcr.io/company/product-service:sha-a81c92d
  resources:
    requests:
      cpu: 250m
      memory: 256Mi
    limits:
      cpu: 500m
      memory: 512Mi
```

In our Helm chart, these values should normally come from values.yaml.

11. Why Horizontal Pod Autoscaling?

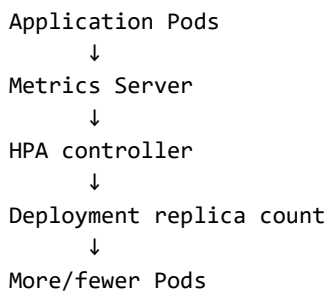
Resource limits protect a Pod, but they do not create more Pods. If traffic increases, a stateless microservice may need additional replicas. HPA changes the desired replica count based on observed metrics.

12. Horizontal vs Vertical Scaling

Approach	What changes?	Example
Horizontal	Number of Pods	2 Pods → 5 Pods
Vertical	Resources per Pod	256Mi → 512Mi memory

For stateless Spring Boot services, horizontal scaling is often a natural Kubernetes strategy.

13. HPA Feedback Loop



The HPA observes metrics and adjusts the desired number of replicas. Kubernetes then creates or removes Pods.

14. What Is Metrics Server?

Metrics Server is a Kubernetes add-on that collects basic resource usage information from Kubelets and exposes it for components such as HPA and commands such as `kubectl top`.

Tool	Typical role	Used here
Metrics Server	Basic CPU/memory resource metrics	HPA and <code>kubectl top</code>
Prometheus	General metrics collection/querying/alerting	Observability later

Metrics Server and Prometheus are not the same tool.

15. Verify Metrics Server

```
kubectl get deployment metrics-server -n kube-system
kubectl top pods -n ecommerce
kubectl top nodes
```

If `kubectl top` does not work, fix the metrics source before troubleshooting the HPA.

16. CPU Utilization and HPA

For a CPU-based HPA, utilization is normally calculated relative to CPU requests. Therefore CPU requests are important when using a CPU utilization target.

17. Create an HPA

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: product-service
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: product-service
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

This example asks Kubernetes to keep at least two replicas, allow up to six, and target approximately 70% average CPU utilization.

18. Important HPA Terms

- `scaleTargetRef` — the Deployment whose replica count HPA changes.
- `minReplicas` — the minimum replica count.
- `maxReplicas` — the maximum replica count.
- `metric` — the measurement used for scaling.
- `averageUtilization` — target average CPU utilization relative to CPU requests.

19. HPA Is a Feedback Loop, Not an Instant Switch

Metrics are collected, HPA evaluates them, the Deployment replica count changes, new Pods are scheduled and started, and readiness checks must pass. Therefore HPA is not an instantaneous response to every traffic spike.

20. Scale-Up and Scale-Down

Kubernetes supports behavior controls and stabilization windows to reduce oscillation. A common idea is to react faster to increasing demand but wait longer before reducing capacity.

```
behavior:
  scaleDown:
    stabilizationWindowSeconds: 300
  scaleUp:
    stabilizationWindowSeconds: 0
```

Treat these as tuning examples. Production values should be based on workload behavior.

21. HPA Limitations

- HPA cannot create new Kubernetes nodes.
- HPA needs usable metrics.
- HPA does not make every stateful workload safe to scale horizontally.
- HPA cannot guarantee zero latency during sudden traffic spikes.
- HPA cannot fix database bottlenecks, connection-pool limits, or application-level bottlenecks.
- HPA does not replace readiness/liveness/startup configuration.

Node autoscaling and deeper workload tuning are separate concerns.

22. Helm Revisited

Session 15 introduced Helm. Session 16 uses Helm to make resource and autoscaling configuration reusable.

```
replicaCount: 2

resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    cpu: 500m
    memory: 512Mi

autoscaling:
  enabled: true
  minReplicas: 2
  maxReplicas: 6
  targetCPUUtilizationPercentage: 70
```

23. Conditional HPA Template

```
{{- if .Values.autoscaling.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ include "product-service.fullname" . }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
```

```

    kind: Deployment
    name: {{ include "product-service.fullname" . }}
    minReplicas: {{ .Values.autoscaling.minReplicas }}
    maxReplicas: {{ .Values.autoscaling.maxReplicas }}
    metrics:
      - type: Resource
        resource:
          name: cpu
          target:
            type: Utilization
            averageUtilization: {{ .Values.autoscaling.targetCPUUtilizationPercentage }}
  {{- end }}

```

The same chart can therefore enable or disable autoscaling per environment.

24. Raw Manifests vs Helm — Design Choice

Raw manifests	Helm	Course rule
Small/static deployment	Repeated patterns across services/environments	Start simple; use Helm when repetition appears.
Maximum direct YAML visibility	Parameterized images/resources/HPA	Prefer Helm for the growing platform.
One-off resource	Reusable packaging and release management	Do not template just for the sake of templating.

25. RBAC — Why Permissions Matter

A Pod can run with a Kubernetes identity called a ServiceAccount. Kubernetes RBAC controls what that identity may do.

Least privilege means: a service receives only the Kubernetes API permissions it actually needs.

26. ServiceAccount

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: product-service
  namespace: ecommerce

```

Deployment example:

```

spec:
  serviceAccountName: product-service

```

27. Role

A Role defines permissions within one namespace.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: product-service
rules:
  - apiGroups: [""]
    resources: ["configmaps"]
    verbs: ["get", "list"]
```

This is namespace-scoped and grants only read access to ConfigMaps in that namespace.

28. RoleBinding

A RoleBinding connects the Role to an identity such as the ServiceAccount.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: product-service
subjects:
  - kind: ServiceAccount
    name: product-service
roleRef:
  kind: Role
  name: product-service
  apiGroup: rbac.authorization.k8s.io
```

29. Role vs ClusterRole

Resource	Scope	Typical use
Role	One namespace	Service-specific permissions
ClusterRole	Cluster-wide or reusable cluster-scoped permissions	Cluster-scoped resources or broader requirements

Do not use ClusterRole simply because it is easier. Prefer namespace-scoped permissions when possible.

30. Common RBAC Verbs

- get
- list
- watch
- create
- update

- patch
- delete

Grant only the verbs required by the application.

31. Does Every Spring Boot Service Need RBAC?

No. Many Spring Boot microservices never call the Kubernetes API. If product-service only talks to PostgreSQL, Redis, Kafka, other services, and external APIs, it may need no Role at all.

The correct rule is: give every workload an explicit identity when appropriate, but grant Kubernetes API permissions only when required.

32. RBAC in Helm

```
serviceAccount:  
  create: true  
  name: product-service
```

```
rbac:  
  create: false
```

Keeping RBAC creation disabled by default is a useful safety pattern. Enable it only when the application actually needs Kubernetes API access.

33. If the Service Needs Kubernetes API Access

Document the exact resources and verbs first. Then create the smallest namespace-scoped Role and RoleBinding needed.

Example: an application that only needs to read ConfigMaps should not receive create/delete permissions or cluster-wide administration.

34. Namespaces and Environment Isolation

Namespaces provide logical separation inside a Kubernetes cluster. For our course:

```
ecommerce-dev  → developer testing  
ecommerce      → integration/staging  
ecommerce-prod → production
```

Namespaces help organize resources and apply policies, but they are not a complete security boundary by themselves.

35. Namespace Commands

```
kubectl create namespace ecommerce  
kubectl get all -n ecommerce
```

```
kubectl config set-context --current --namespace=ecommerce
kubectl get pods -n ecommerce
```

36. ResourceQuota and LimitRange — Awareness

Kubernetes also provides ResourceQuota and LimitRange. ResourceQuota can limit aggregate namespace consumption. LimitRange can provide defaults or constraints for individual resources. These mechanisms complement Pod-level requests and limits.

37. Production-Oriented Product Service Values

```
replicaCount: 2

image:
  repository: ghcr.io/company/product-service
  tag: sha-a81c92d
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 8080

resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    cpu: 500m
    memory: 512Mi

autoscaling:
  enabled: true
  minReplicas: 2
  maxReplicas: 6
  targetCPUUtilizationPercentage: 70

serviceAccount:
  create: true
  name: product-service

rbac:
  create: false
```

38. Spring Boot Readiness + HPA

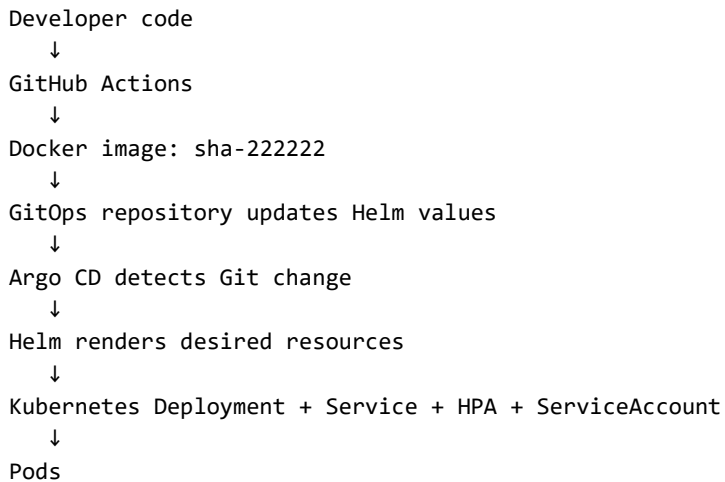
HPA may create new Pods, but traffic should only reach Pods that are ready. A Spring Boot application may need time to start the JVM, initialize the application context, connect to dependencies, and become ready.

- Resource requests/limits

- Readiness probe
- Appropriate startup/liveness strategy
- HPA
- Rolling update settings
- Application metrics and logs

39. GitOps Integration

With Sessions 14–15, Helm configuration belongs in the GitOps desired state.



40. End-to-End Architecture



41. Hands-On Lab

Goal: make product-service production-aware by adding resources, HPA, Helm configuration, and workload identity. Implement RBAC only when the service actually needs Kubernetes API access.

Lab Task 1 — Inspect Current Resources

```
kubectl get pods -n ecommerce
kubectl describe pod <pod-name> -n ecommerce
kubectl top pods -n ecommerce
```

Record current CPU/memory usage and identify what is missing from the Deployment.

Lab Task 2 — Add Resource Requests and Limits

```
resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    cpu: 500m
    memory: 512Mi
```

```
helm template product-service ./product-service -n ecommerce
helm upgrade --install product-service ./product-service -n ecommerce
```

Lab Task 3 — Verify Resources

```
kubectl describe pod <pod-name> -n ecommerce
```

Verify the resource requests and limits in the container configuration.

Lab Task 4 — Verify Metrics Server

```
kubectl top pods -n ecommerce
kubectl top nodes
```

Do not continue with HPA troubleshooting until usable metrics are available.

Lab Task 5 — Add HPA Values

```
autoscaling:
  enabled: true
  minReplicas: 2
  maxReplicas: 6
  targetCPUUtilizationPercentage: 70
```

Lab Task 6 — Add HPA Template

Create templates/hpa.yaml using autoscaling/v2. Make sure it targets the exact Deployment name generated by the Helm chart.

```
helm template product-service ./product-service -n ecommerce
```

Lab Task 7 — Deploy and Inspect HPA

```
helm upgrade --install product-service ./product-service -n ecommerce
kubectl get hpa -n ecommerce
kubectl describe hpa product-service -n ecommerce
```

Lab Task 8 — Generate Load

Generate sustained requests against Product Service using the load-testing method available in your environment. The goal is to raise CPU enough for HPA to react.

```
kubectl get hpa -n ecommerce -w
kubectl get pods -n ecommerce -w
kubectl top pods -n ecommerce
```

Observe that HPA changes the desired replica count and Kubernetes creates/removes Pods.

Lab Task 9 — Explain the Feedback Loop

Students must explain: CPU increases → Metrics Server reports usage → HPA calculates desired replicas → Deployment replica count changes → Kubernetes creates Pods → readiness checks pass → more capacity becomes available.

Lab Task 10 — Add a Dedicated ServiceAccount

Create the ServiceAccount through Helm and reference it from the Deployment.

```
kubectl get serviceaccount -n ecommerce
```

Lab Task 11 — Make the RBAC Decision

Ask first: Does product-service actually need to call the Kubernetes API?

If NO: keep `rbac.create=false` and grant no unnecessary Kubernetes API permissions.

If YES: document the required resources and verbs, then create a namespace-scoped Role and RoleBinding.

Lab Task 12 — Test RBAC

```
kubectl auth can-i get configmaps --as=system:serviceaccount:ecommerce:product-service -n ecommerce
```

```
kubectl auth can-i delete deployments --as=system:serviceaccount:ecommerce:product-service -n ecommerce
```

The result must match the exact permissions defined by the Role.

Lab Task 13 — GitOps Commit

Commit the Helm changes to the GitOps repository using the course convention:

```
session-16: add-kubernetes-resources-hpa-rbac
```

Push the change and observe Argo CD detect the desired-state update.

42. Troubleshooting Guide

HPA shows <unknown> CPU — Check Metrics Server, verify CPU requests exist, and inspect HPA events.

HPA never scales — Check current CPU, target threshold, min/max replicas, metrics, and whether the workload actually consumes CPU.

Pods are OOMKilled — Memory limit may be too low or the application may have a memory problem. Inspect events and JVM behavior.

Pods are Pending — Inspect `kubectl describe pod`. Requests may exceed node capacity or scheduling constraints may prevent placement.

Helm install fails — Run `helm lint` and `helm template` first and inspect the rendered YAML.

RBAC returns Forbidden — Check ServiceAccount, Role, RoleBinding, namespace, resource, and verb.

Argo CD reverts a manual change — That is expected for GitOps-managed resources. Change the Git desired state instead.

43. Common Student Mistakes

- Confusing requests with limits.
- Using arbitrary resource values without measuring the application.
- Expecting HPA to work without usable metrics.
- Assuming HPA creates Kubernetes nodes.
- Assuming HPA reacts instantly to every spike.
- Creating ClusterRole permissions when a namespace-scoped Role is enough.
- Giving a microservice administrator permissions for convenience.
- Creating RBAC permissions even when the application never calls the Kubernetes API.
- Editing Argo CD-managed resources directly with `kubectl`.
- Using Helm templates without checking the rendered YAML.

44. Quick Quiz

1. What is a CPU request?

Answer: The CPU amount Kubernetes uses for scheduling/resource planning.

2. What can happen when CPU exceeds a container limit?

Answer: CPU can be throttled.

3. What can happen when memory exceeds a container limit?

Answer: The container may be terminated and reported as OOMKilled.

4. Why are CPU requests important for CPU-based HPA?

Answer: CPU utilization targets are calculated relative to CPU requests.

5. What does HPA change?

Answer: The desired replica count of a scalable workload.

6. What is Metrics Server?

Answer: A Kubernetes component exposing basic resource usage such as CPU and memory.

7. Horizontal vs vertical scaling?

Answer: Horizontal changes Pod count; vertical changes resources per Pod.

8. What are minReplicas and maxReplicas?

Answer: The lower and upper bounds for HPA scaling.

9. What is a ServiceAccount?

Answer: An identity used by a workload running in Kubernetes.

10. What does a Role define?

Answer: Permissions within a namespace.

11. What does RoleBinding do?

Answer: Associates a Role with an identity such as a ServiceAccount.

12. Should every Spring Boot service receive Kubernetes API permissions?

Answer: No. Only services that need the Kubernetes API should receive them.

13. Why use Helm?

Answer: To package and parameterize Kubernetes configuration and reuse it across environments/services.

45. Assessment

Standard Lab Rubric: Feature 70% + Tests/Verification 20% + Quality 10%

- 20% — Correct resource requests and limits.
- 20% — Working HPA with correct minimum/maximum replicas and CPU target.
- 10% — Demonstrated metrics and scaling behavior.
- 10% — Helm parameterization of resources and autoscaling.
- 10% — Correct ServiceAccount configuration.
- 10% — RBAC decision and least-privilege implementation where applicable.

- 10% — Tests/verification using kubectl and Helm.
- 10% — Clean chart structure and Git commit discipline.

46. Homework

Extend the Product Service Helm chart for ecommerce-dev and ecommerce-prod. Use different resource requests/limits and HPA min/max values. Use an immutable image tag. Keep RBAC disabled unless the service actually needs Kubernetes API access. Commit the changes to the GitOps repository and verify that Argo CD can synchronize the desired state.

47. Instructor Guide

Start with the traffic-spike story rather than YAML. Ask students what happens if every Pod has unlimited resources. Then introduce requests and limits.

Next ask: 'What if two replicas are healthy but traffic becomes 10x higher?' This naturally introduces HPA.

Before writing HPA YAML, verify Metrics Server and kubectl top. This prevents students from debugging an HPA with no metrics source.

For RBAC, emphasize the opposite of a common beginner habit: do not create permissions simply because Kubernetes provides them. First ask whether the application needs Kubernetes API access.

Finish by connecting everything:

CI → Image → GitOps → Helm → Kubernetes → Resources → HPA → Secure workload identity

48. Session 16 Takeaway

A Kubernetes deployment becomes production-oriented when we stop thinking only about 'running a Pod' and start thinking about resource control, automatic scaling, repeatable packaging, and least-privilege access.

Requests/Limits → resource expectations and control

HPA → replica scaling

Metrics Server → basic resource metrics

Helm → packaging and parameterization

ServiceAccount → workload identity

RBAC → Kubernetes API permissions

Argo CD → Git-to-cluster reconciliation

The platform can now move from a basic Kubernetes deployment toward a more controlled, scalable, and repeatable runtime model.